The Road to Composable Data Systems: Thoughts on the Last 15 Years and the Future

September 1, 2023

RETROSPECTIVE THOUGHTS

Wes McKinney

A new joint VLDB paper on Composable Data Management Systems with Meta, Databricks, Sundeck, and others at is out! This post is a reflection on how I arrived at thinking about these problems and what the future might look like. Enjoy.

Getting Started: 2008 to 2015

I started building data analysis tools a little over fifteen years ago, in April 2008. The world has changed a lot since then. Going back to the late 2000s, what I perceived at that time was the urgent "Pythonification" of data science. This was as much about making data science more accessible for a large new generation of data practitioners as it was about making existing data scientists more productive. In a 2011 blog post, I wrote about the need for better coordination among Python projects to create a more productive end-to-end user experience, at one point asserting that "fragmentation is killing us."

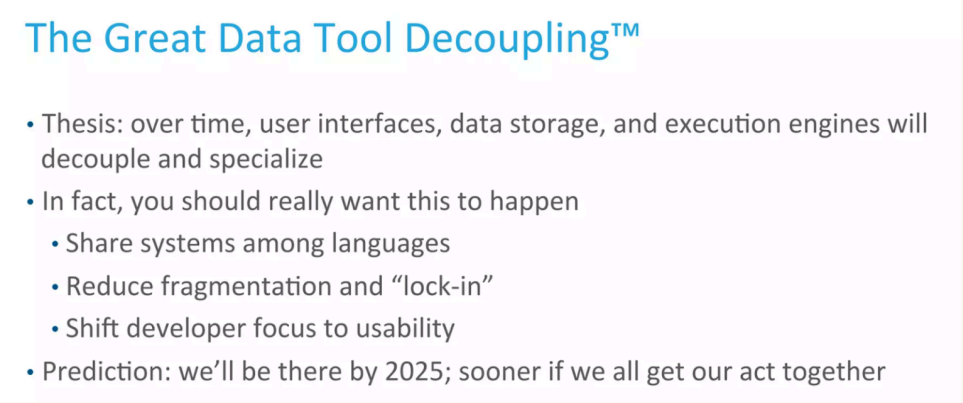
By 2013, pandas was successful enough for me to hand over the reins to the other core developers, led initially by Jeff Reback. Over the last ten years, they have done an incredible job growing the project and its user community. Around that time, Jupyter had also become established along with other essential projects like scikit-learn and statsmodels. My book *Python for Data Analysis* had just been published. Things were looking good for the "Pythonification" story, though we didn't know just how popular Python would become due to the incoming wave of machine learning, data engineering, and distributed computing frameworks.

When Chang She and I started working on DataPad, a visual analytics startup, we quickly started experiencing some of pandas's limitations for building a large-scale, multi-tenant cloud analytics service. pandas had performance, scale, and memory use problems that affected us greatly, even though most pandas users were perfectly happy since few people really had "big data". Aside from performance and resource use issues, interoperability and composability with other systems (e.g. other storage or computing platforms) was another pain point. While prototyping a ground-up redesign of pandas, I began to dwell on the more foundational problems that led to the quandary that we were in. I expressed some of these frustrations in my viral PyData NYC 2013 talk "10 Things I Hate About pandas" (note: I still love pandas!).

The TL;DR is that I felt there were several interconnected issues: pandas had accumulated rough edges from its relationship with NumPy, and NumPy was intended for numerical computing, not building databases. pandas solved many problems that database systems also solve, but almost no one in the data science ecosystem had the expertise to build a data frame library using database techniques. Eagerly-evaluated APIs (as opposed to "lazy" ones) make it more difficult to do efficient "query" planning and execution. Data interoperability with other systems is always going to be painful unless faster, more efficient "standards" for interoperability are created.

Another way I look at this with fifteen years of hindsight is that pandas had to do everything for itself, and this is an enormous burden for a fully volunteer-based open source project. Things like language-independent data interoperability standards or plug-and-play components for efficient query processing were pie-in-the-sky ideas then but only now have become more realistic.

When the DataPad team and I joined Cloudera in late 2014, my horizons started to expand, and it started seeming more feasible to pursue solutions to some of these deeper infrastructure problems. By early 2015, I was talking openly about this, describing a vision for better interoperability, composability, and reuse amongst database- and data frame-like systems. In one April 2015 talk, I described this as the "Great Data Tool Decoupling" (this certainly wasn't an idea that I came up with, since storage "disaggregation" was one of the *raisons d'être* of the Hadoop ecosystem):



(See video from this talk from 4:20 to 6:35)

> *Thankfully, around this time, there was a sort of collective awakening to the need for cross-cutting solutions to many of these kinds or problems. Since it's 2023 now, and I predicted we'd "be there by 2025", I'll spend the rest of the post discussing why this work has been happening now and some of what's been done to help advance us toward these goals. I'll conclude with some thoughts about where I think we need to invest in the coming years to accelerate the next wave of progress.*

Why Now? The Changing Hardware Landscape

Many people have asked why so many of us have become concerned with these problems of interoperability, modularity, composability, decoupling, and so on. Julien Le Dem, who co-designed the Parquet file format and later co-designed Apache Arrow with us, coined the term "deconstructed database" to describe the process of decomposing vertically-integrated systems into modular, reusable components.

There are many root causes, but one of the main ones is the evolution of computing hardware. This has manifested in faster storage devices, faster networking, and both faster and more specialized processors. Faster storage and networking means that we must rethink how we store, access, and move around large datasets that we need to process in different kinds of systems. The rapid development of many-core CPUs and specialized processors like GPUs and TPUs means that we must rethink the programming interface to enable developers to use cutting-edge hardware without needing a Ph.D. in Computer Engineering. Computational pipelines in real-world

applications have become more heterogeneous as well, both in the kinds of data processing and programming languages used. From this perspective, it has become increasingly suboptimal to build each processing component in the traditional vertically-integrated fashion.

The ML/AI ecosystem was one of the first movers to enable developers to seamlessly take advantage of hardware acceleration through user-facing frameworks like TensorFlow and PyTorch, assisted by new compiler technologies like XLA, MLIR, and JAX. Chris Lattner, the creator of LLVM, MLIR, and Swift, has even started a new company aptly named Modular aiming to continue innovating in compiler technology that assists with hardware heterogeneous computing and developer productivity for AI.

Toward a more Modular, Composable Data Stack

Building a successful standalone software project is hard enough. It can be even more difficult to build things that require coordination and agreement amongst multiple software projects. Out of this awakening emerged many new open source initiatives. I want to highlight some of them not because I think they are the ultimate solutions but because they are helping light the way and show what the future may look like:

- Apache Arrow (2015) — a language-independent compute and data interchange layer and supporting systems infrastructure across important programming languages
- Ibis (2014) and dplyr (2012) — backend-agnostic data frame interfaces
- RAPIDS (2018) — GPU-accelerated libraries for data analytics and machine learning
- DuckDB (2018) and Velox (2021) — embeddable systems providing fast columnar query processing
- Substrait (2021) — a language-independent intermediate representation (IR) middleware for analytical computing to assist in decoupling user interfaces from compute engines

Together these projects are working to enable modularity in data interchange, query execution, and programming interfaces. I'll say a few things about each of them briefly

and then wrap up with some thoughts about the next several years of open source engineering work in this domain.

Structured Data Interchange and Compute Fabric: Apache Arrow

Over the seven years since its founding, Arrow has had a profound impact on the data analytics world. It has been adopted as a language- and hardware-independent fast interchange and computation layer, and I believe now more than ever that "the future is Arrow-native." Without a standardized solution for data interchange and in-memory computation, systems pay a steep penalty both in computational cost and development time to interoperate with each other. More recent projects in Arrow like, Flight, Flight SQL, and ADBC are paving the way for databases and distributed systems to standardize how they can interact with external systems using the Arrow memory format.

In addition to streamlining data interchange and composing systems with each other, another major benefit of Arrow is that it provides a stable target for hardware vendors to develop accelerated "kernels" for doing analytics on their hardware. In numerical computing and linear algebra, we have come to take hardware-optimized kernel libraries like Intel's MKL (now oneMKL) for granted. Arrow enables the same thing to take place for analytics and database operations through projects like RAPIDS and embeddable database engines like the ones discussed below.

In 2017 I wrote about why I thought that Arrow would provide many of the solutions to the "10 Things I Hate About pandas". Nearly 6 years later, I would say that overall things have gone about as well as I could have hoped. New, high-performance data frame libraries like Polars are built on Arrow, and pandas has retrofitted itself in many places with Arrow-based acceleration. It's been rewarding to see these improvements trickle down to other projects like Dask.

Hardware Acceleration: RAPIDS

Since NVIDIA's introduction of CUDA in 2006, developers have had access to a set of low-level tools enabling general-purpose computing on GPUs. The parallel architecture of GPUs (thousands of processing cores per device) provides much

higher throughput than CPUs (several dozen cores at most). In the decade following the introduction of CUDA, developers built a patchwork of open source software that could take advantage of GPU hardware to accelerate some types of data analytics and machine learning workloads, especially deep learning training and inference. But broader adoption of GPUs was slowed by the need to program in C/C++ to use CUDA.

Seeing an opportunity to speed adoption of GPU acceleration in data applications, NVIDIA in 2016 began to develop a suite of domain-specific open source libraries built on top of CUDA. Released in 2018 as RAPIDS, this suite of libraries—including CUDA DataFrames (cuDF), CUDA MachineLearning (cuML), and many others—were built to accelerate data processing for analytics and machine learning using the Arrow memory format internally. Their Python APIs were designed to maximize similarity to analogous existing Python libraries (cuDF mirrors pandas; cuML mirrors scikit-learn).

RAPIDS was quickly adopted by many Python users and by projects including Dask and BlazingSQL.

Modular Query Processing: DuckDB, Velox, and friends

Analytic database users have had access to extremely fast performance since the first wave of analytic DBMSs like Vertica, Vectorwise, SAP HANA, and many others developed starting in the mid- to late-2000s. The cutting-edge research that yielded these commercial systems led directly to the modern DBMS technologies popular today such as BigQuery, Clickhouse, Databricks SQL, Redshift, or Snowflake. Unfortunately, there have been few experts capable of designing and implementing such systems and much of the software created has been squirreled away inside closed-source commercial products.

An incredible thing has happened in recent years with the emergence of new embeddable columnar database engines like DuckDB, Velox, and DataFusion (part of the Arrow Rust project). Even better, these projects are designed to be used in systems that are Arrow-based. The ability to add cutting-edge analytic SQL processing to almost any application is a disruptive and transformative change for our industry. CMU database professor Andy Pavlo made the bold prediction that "the commoditization of these query execution components means that all OLAP DBMSs

will be roughly equivalent in the next five years… [E]very DBMS will have the same vectorized execution capabilities that were unique to Snowflake ten years ago. ” I believe he’s right and this is a big deal.

Modular Programming Interfaces: Ibis, dplyr, and others

At the same time that I was helping start the Arrow project, I also went back to the drawing board on the programmer interface for interacting with databases and data frame engines. My attempt at marrying SQL and data frame APIs was Ibis which has been churning along and growing for more than seven years now. Rather than cloning the pandas API, I worked with Phillip Cloud and others to design a lazily-evaluated expression system that is pleasant to use, extensible, and can support a broad set of SQL-like and non-SQL-like data processing use cases. We wanted to have complete coverage of SQL capabilities but with strong type-checking and easy reuse of expression fragments to enable developers to be much more productive in authoring complex analytics workloads that are also portable across different execution engines and processing frameworks.

In the last couple of years, we have made major improvements to Ibis’s internals and expanded support to 16 different execution backends. We are working to make Ibis the ultimate database analytics API for enterprises using Python. It’s exciting to see the project being adopted for database API projects within Google Cloud, Microsoft, and other companies.

Similar trends in modular, portable data frame APIs have been happening elsewhere, too. When starting to work on Ibis, I was partly inspired by R’s dplyr and tidyverse projects, which kicked off in 2012. More recently in the Python ecosystem, the Modin project has worked to bring more rigor and formalism to data frame operations by defining an expanded relational algebra to enable a large subset of pandas code to be compiled and executed on a distributed cluster.

As an aside, I’m also really excited about efforts to create entire new query languages that compile to SQL, like Malloy and PRQL.

One challenge to building and maintaining libraries like Ibis and dplyr or new query languages like Malloy is that they must implement engine-specific backend interfaces that generate a particular SQL dialect or another query representation, which means a lot of additional implementation complexity, opportunity for bugs and inconsistencies, and so on. Wouldn't it be great if there were a standardized way to manipulate and transport analytical query expressions?

An "IR" for Data Analytics: Substrait

SQL is often thought of as a standard for analytical queries, but in practice, each SQL database has developed its own slightly-different query language so that few SQL dialects are portable in practice. Switching to a new database generally means rewriting a lot of queries and learning new syntax for advanced data manipulation (e.g. for working with nested data). This fragmentation of representing queries and data manipulation expressions has hampered efforts to enable modularity or interoperability at this level.

In more recent times, the popularity of the cloud "lakehouse" architecture and the need to utilize different execution engines on top of common datasets has made the query persistence and interoperability problem even more painful. Companies like Airbnb and LinkedIn have gone so far as to develop SQL transpiler projects like sqlglot and Coral.

Recognizing the unsustainability of these trends, a group of developers led by Jacques Nadeau (who was also a key Arrow developer) started Substrait to define a standardized low-level intermediate representation for queries to enable systems to engage with each other independent of their own native SQL dialect or other query languages.

Substrait is one of the more esoteric and least user-facing projects in this post, but I believe it is an essential part of enabling execution engines to become more modular and composable and to make it easier for "frontend" frameworks like Ibis or dplyr to focus on providing a pleasant and productive user experience without getting bogged down in the quirks of supporting each backend target.

What's missing? Query Optimization, Distributed Computing, File Formats, Dataset Management

A few areas that I have omitted from this post include modular query optimization (like Apache Calcite), distributed execution frameworks (like Dask or Ray for Python), file formats (like Parquet or ORC), and large-scale dataset management tools (like Apache Iceberg or Delta Lake). These are important parts of the overall story.

My hope is that distributed execution frameworks can develop robust support for data in Arrow format with modular engine support using Substrait as the fungible IR for query fragments that move between schedulers and the backend engines.

I think we will eventually see the adoption of next-generation file formats that provide even better performance on modern storage and processing hardware, but the now "legacy" columnar formats Parquet and ORC are going to be with us for a long, long time.

The new scalable dataset management frameworks like Iceberg and Delta Lake can be adapted to support new file formats in the future, and so to a certain extent file formats and dataset management are orthogonal to many of the issues discussed above.

With the recent explosive growth of AI systems, I expect we will also see innovation in specialized storage and data access technologies to speed up ML and generative AI applications. Lance is one new project here I'm excited about.

Conclusions and Looking Ahead

I'm optimistic about what the future holds for the continuing trends of modularization, interoperability, and composability, and what I dubbed the "Great Decoupling." As an open source community, we have many years of incremental work to do on the above projects to fully realize their potential, but it's been energizing to see developers come together and back these critical open source standardization efforts.

The "commoditization" of query execution through projects like DuckDB, Velox, and DataFusion is surely to be a disruptive trend in the ecosystem as we begin to take for

granted that nearly any system can equip itself with cutting-edge columnar query processing capabilities. Furthermore, ongoing improvements in these modular, reusable components will become more or less instantly available in all of the projects that depend on them. It's super exciting.

Lastly, as developer energy shifts away from "who can build the fastest execution engine", I predict the coming years will bring a new wave of investment in user interface productivity (represented by such projects as Ibis, dplyr, and Malloy) as once again the human-in-the-loop (or LLM-in-the-loop) programmer becomes a bottleneck in getting things done.

I am looking forward to what comes next, and I will surely revisit these topics in the future as things progress.

If you missed it above, check out our VLDB 2023 paper on these topics!